今日及接下来几次课程的内容安排

- 分支预测

回顾：如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。

- 如果指令是控制流指令，潜在的解决方案有：

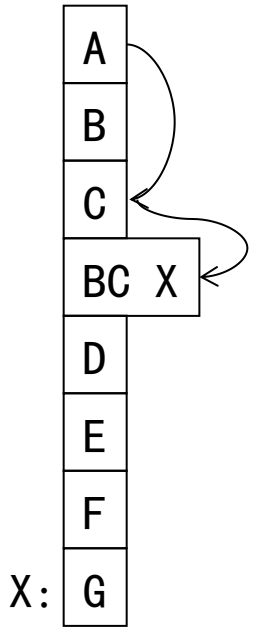
- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行 predicated execution）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

回顾：延迟分支（I）

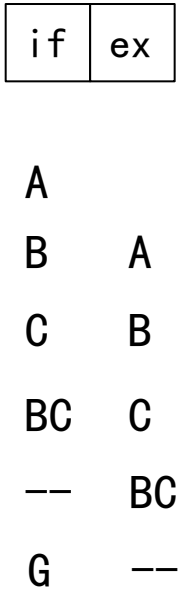
- 改变分支指令的语义
  - 在 N 条指令后分支
  - 在 N 个周期后分支
- 概念：延迟执行分支。分支后面的 N 条指令（延迟槽）无论分支方向如何，始终执行。
- 问题：如何找到填充延迟槽的指令？
  - 分支必须独立于延迟槽指令
- 无条件分支：更容易找到指令来填充延迟槽
- 条件分支：条件计算不应依赖于延迟槽中的指令 → 很难填充延迟槽

回顾：延迟分支（II）

Normal code:

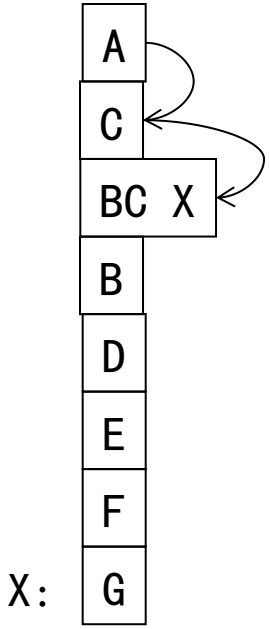


Timeline:

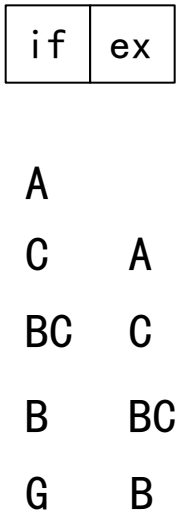


6 cycles

Delayed branch code:



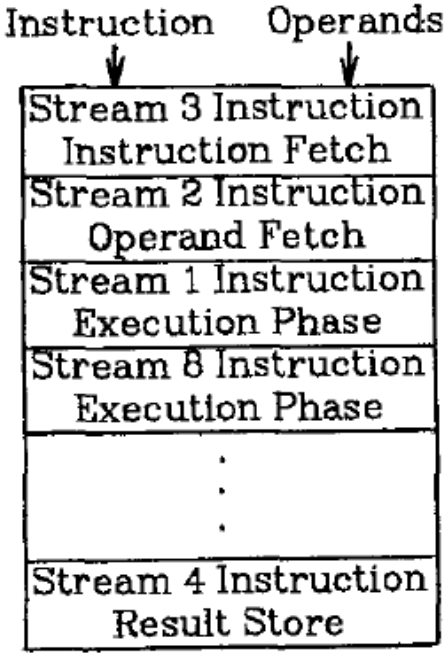
Timeline:



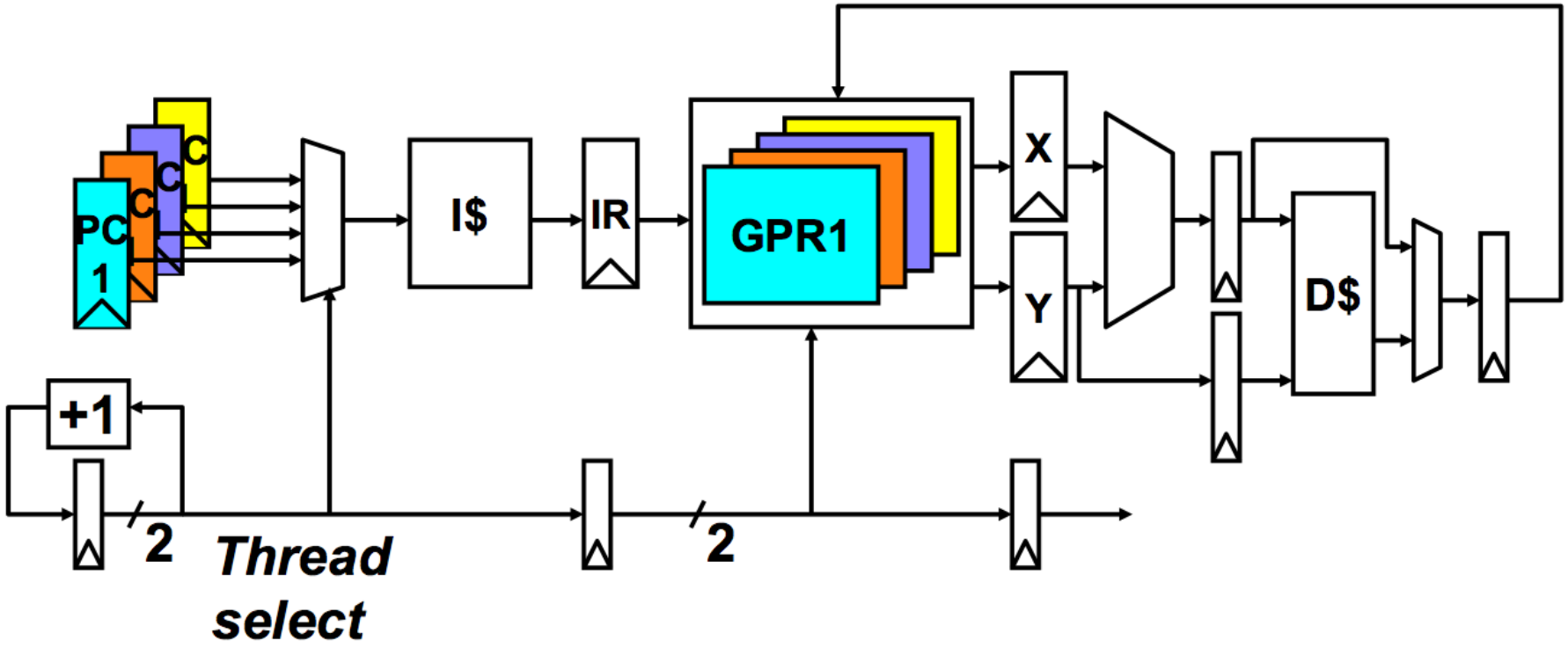
5 cycles

回顾：细粒度多线程

- 概念：硬件具有多个线程上下文。每个周期，取指引擎从不同的线程中取指。
  - 当取到的分支/指令解决时，来自同一线程的指令未被取出
  - 分支/指令解决的延迟与其他线程指令的执行重叠
- 优点：
  - 不需要处理线程内的控制和数据依赖
  - 不同线程的执行不会互相干扰
- 缺点：
  - 单线程性能下降
  - 需要额外的逻辑来保持线程上下文
  - 如果线程不足，无法覆盖整个流水线，则不能重叠延迟



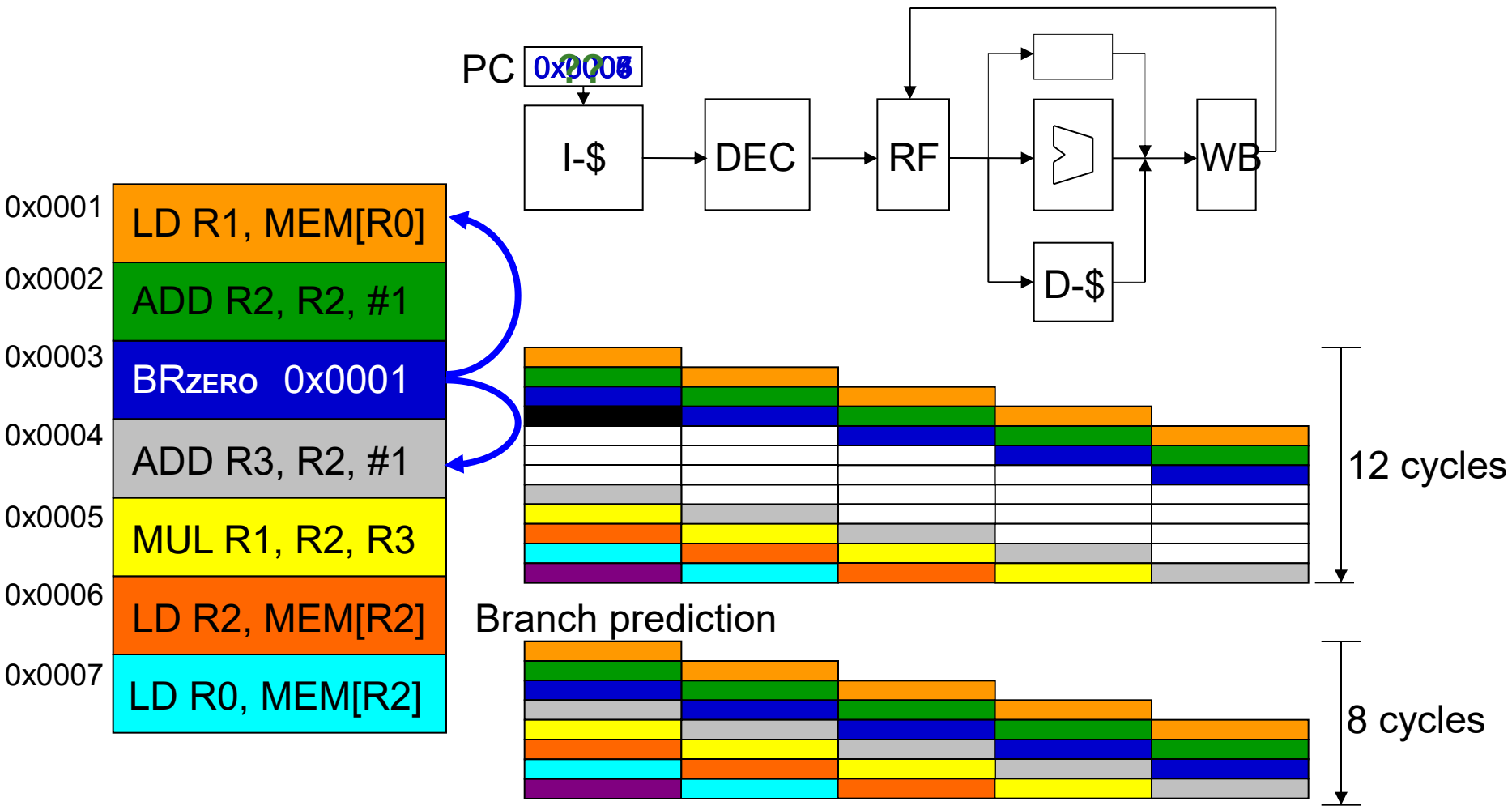
回顾：多线程流水线示例



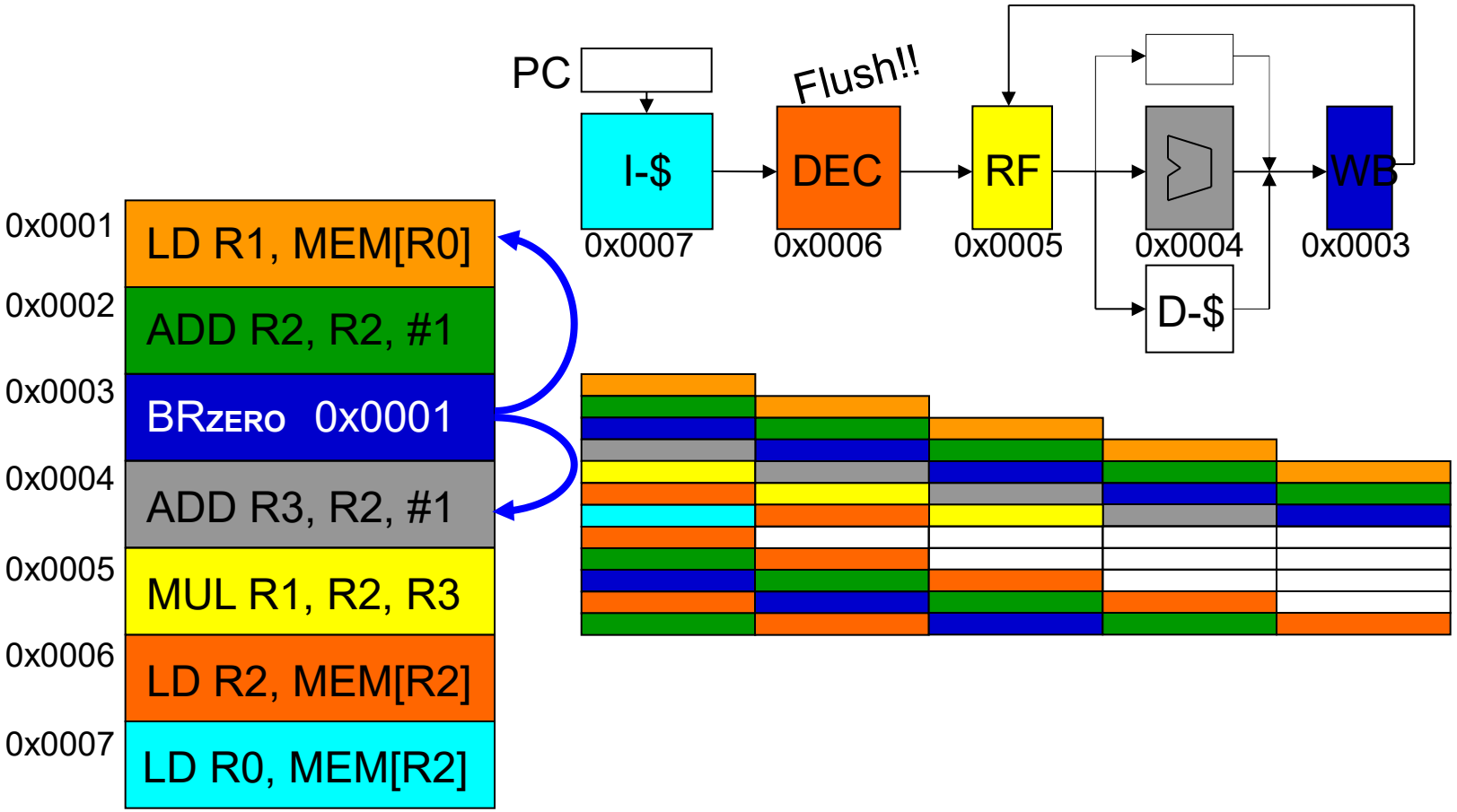
如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。
- 如果指令是控制流指令，潜在的解决方案有：
- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

回顾：分支预测：猜测要获取的下一条指令

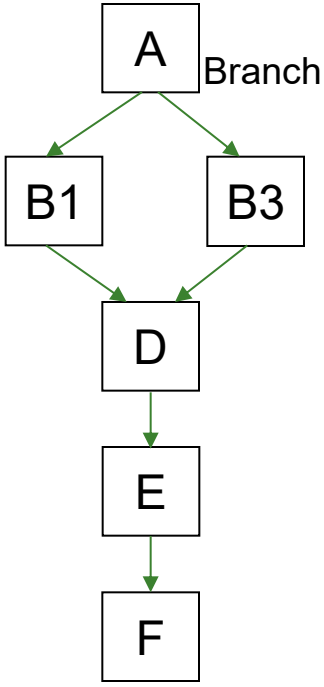


回顾：误判处罚

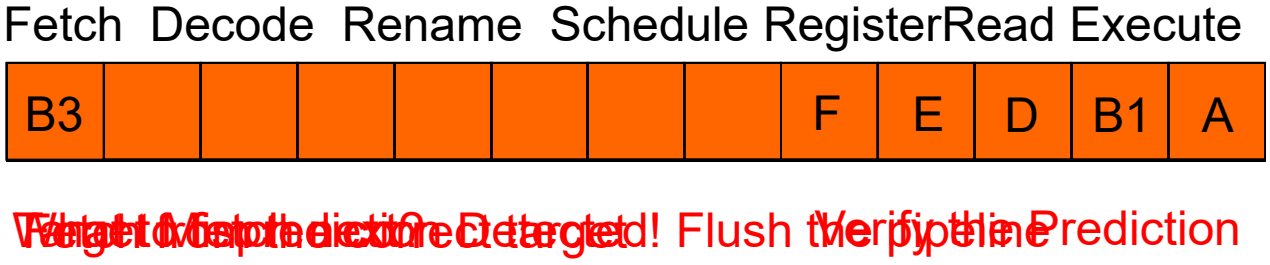

PC
Flush!!
I-$
DEC
RF
WB
D-$
0x0007
0x0006
0x0005
0x0004
0x0003
0x0001
LD R1, MEM[R0]
0x0002
ADD R2, R2, #1
0x0003
BRZERO 0x0001
0x0004
ADD R3, R2, #1
0x0005
MUL R1, R2, R3
0x0006
LD R2, MEM[R2]
0x0007
LD R0, MEM[R2]

回顾：分支预测

- 处理器采用流水线技术来提高并发性。
- 在存在分支指令的情况下，如何保持流水线满负荷运行？
  - 当分支指令被获取时，需要猜测下一条指令。
  - 这需要猜测分支指令的方向和目标。



Pipeline

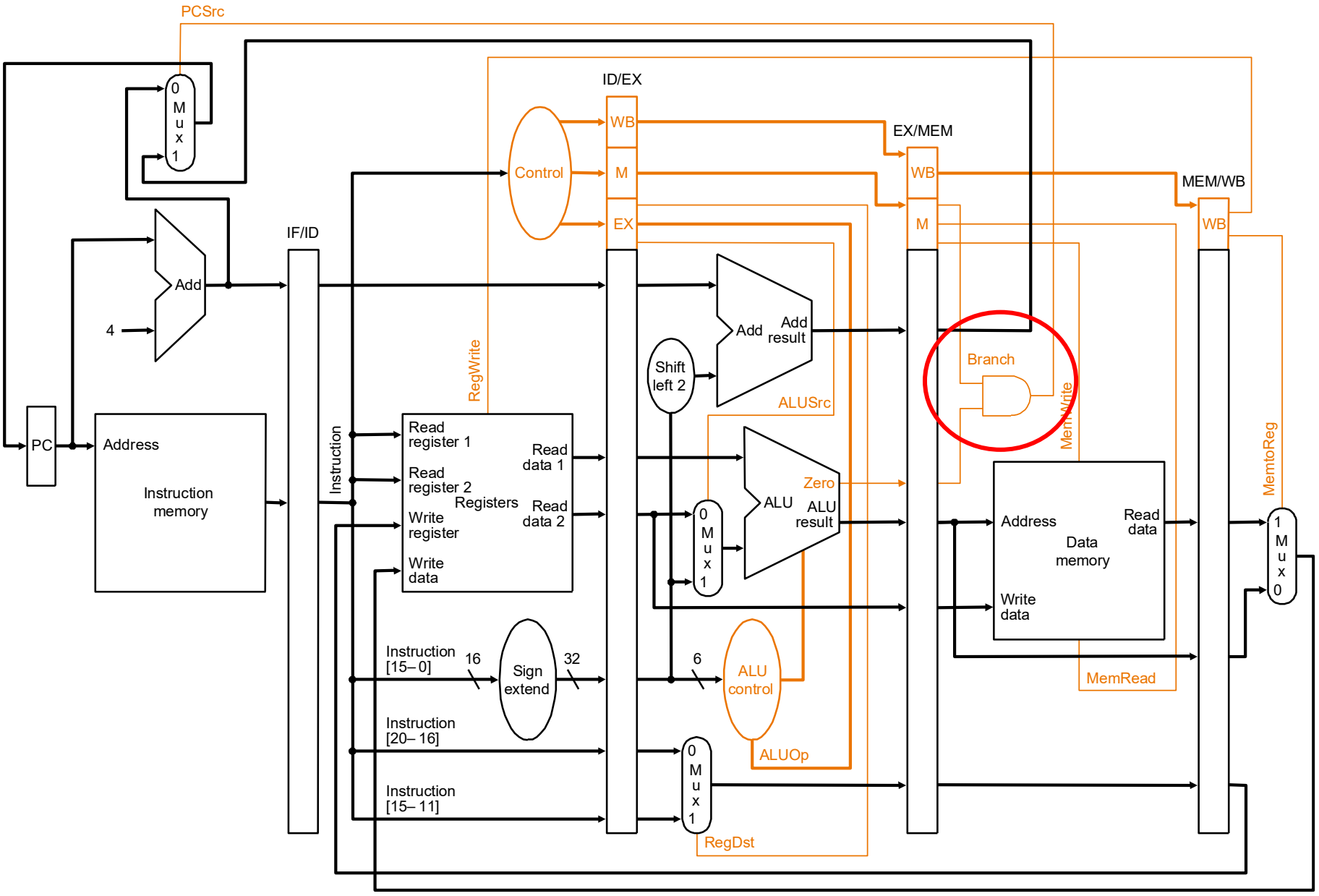


回顾：性能分析

- 猜测正确⇒ 无惩罚 ~86% 的时间
- 猜测错误 ⇒ 2 气泡
- 假设
  - 无数据依赖相关的停顿 (no data dependency related stalls)
  - 20% 的控制流指令
  - 70% 的控制流指令被执行
  - CPI = [1 + (0.20*0.7) * 2] =
    = [1 + 0.14 * 2] = 1.28

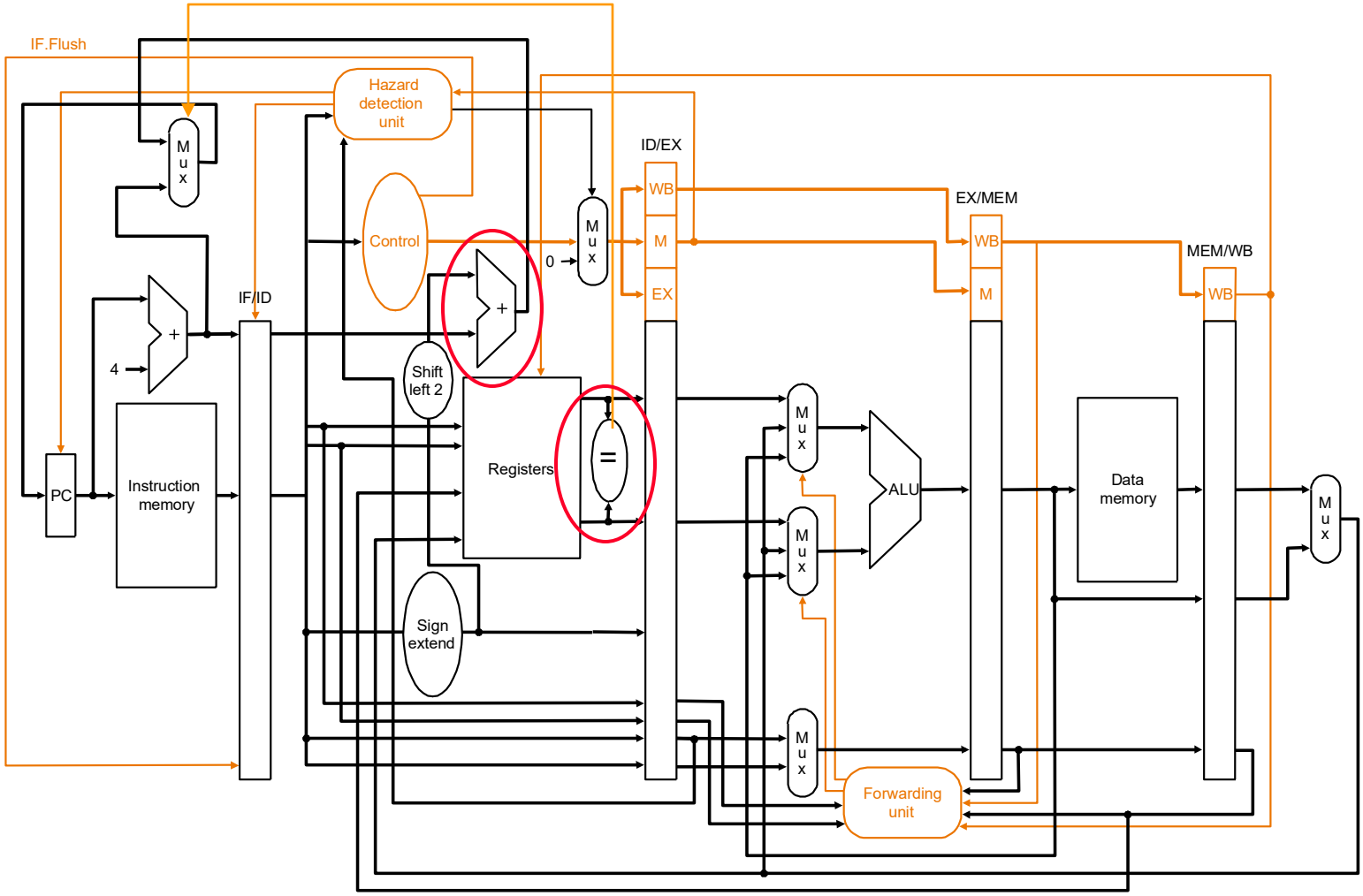
猜错的概率　　错误猜测的惩罚

我们能否减少这两项罚款中的任何一项？


PCSrc
0
M
u
x
1
ID/EX
EX/MEM
MEM/WB
WB
M
EX
Control
IF/ID
Add
4
PC
Address
Instruction memory
Instruction
RegWrite
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALUSrc
Branch
MemWrite
Zero
ALU
ALU result
Address
Data memory
Read data
Write data
MemRead
MemtoReg
Instruction [15–0]
16
Sign extend
32
6
ALU control
ALUOp
Instruction [20–16]
Instruction [15–11]
RegDst

回顾：降低分支预测错误惩罚

- 尽早解决分支条件并确定目标地址



CPI = [1 + (0.2*0.7) * 1] = 1.14

分支预测（进阶）

- Idea：预测下一条取指地址（将在下一周期中使用的）。

- 在取指阶段需要预测三项内容：
  - 判断当前指令是不是分支指令
  - 预测分支方向（Taken? Not taken?）
  - 预测分支目标地址（如果分支被采纳）

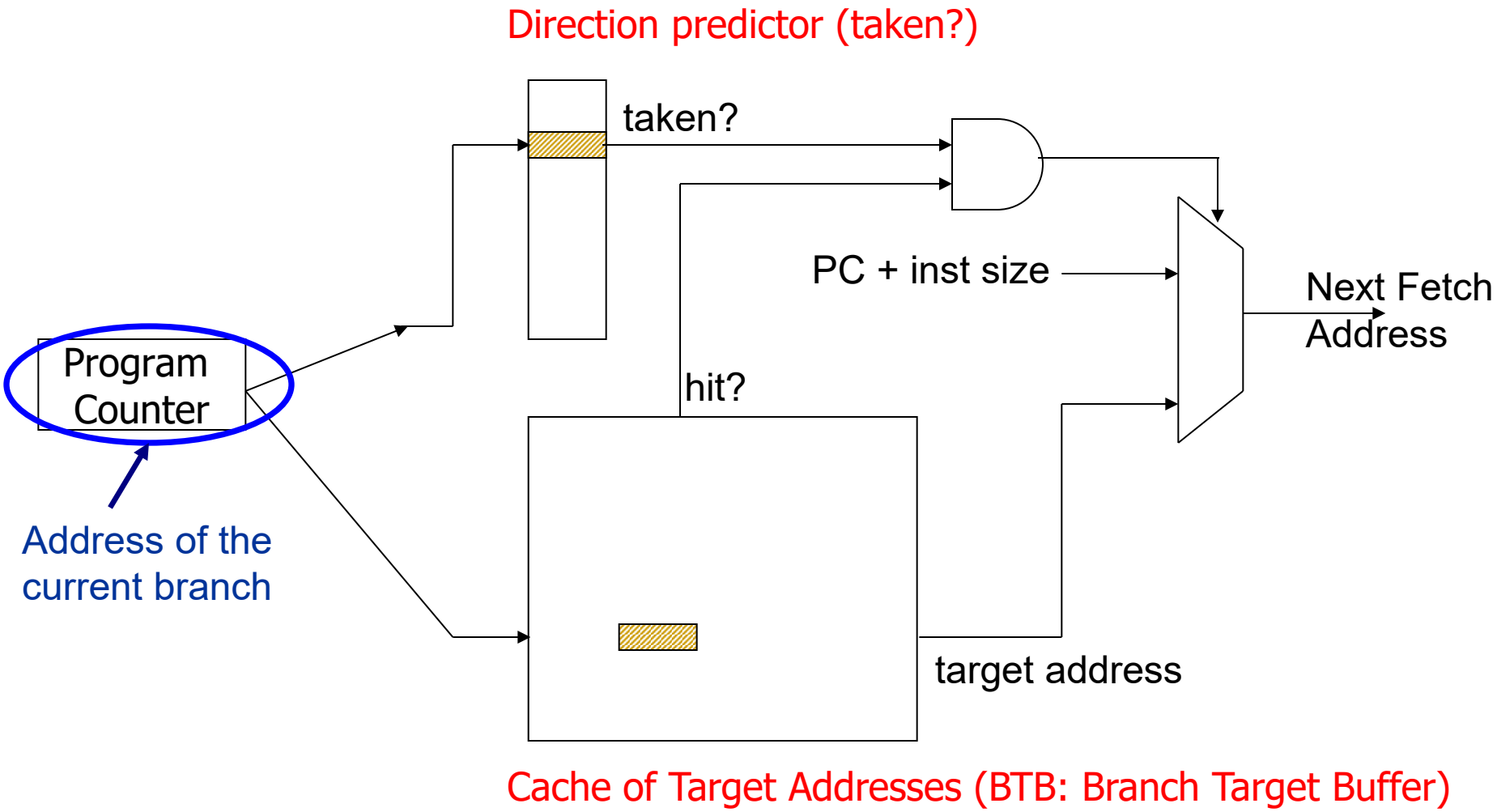
分支预测（进阶）

- **Idea：预测下一条取指地址（将在下一周期中使用的）。**

- **在取指阶段需要预测三项内容：**
  - 判断当前指令是不是分支指令
  - 预测分支方向（Taken? Not taken?）
  - 预测分支目标地址（如果分支被采纳）

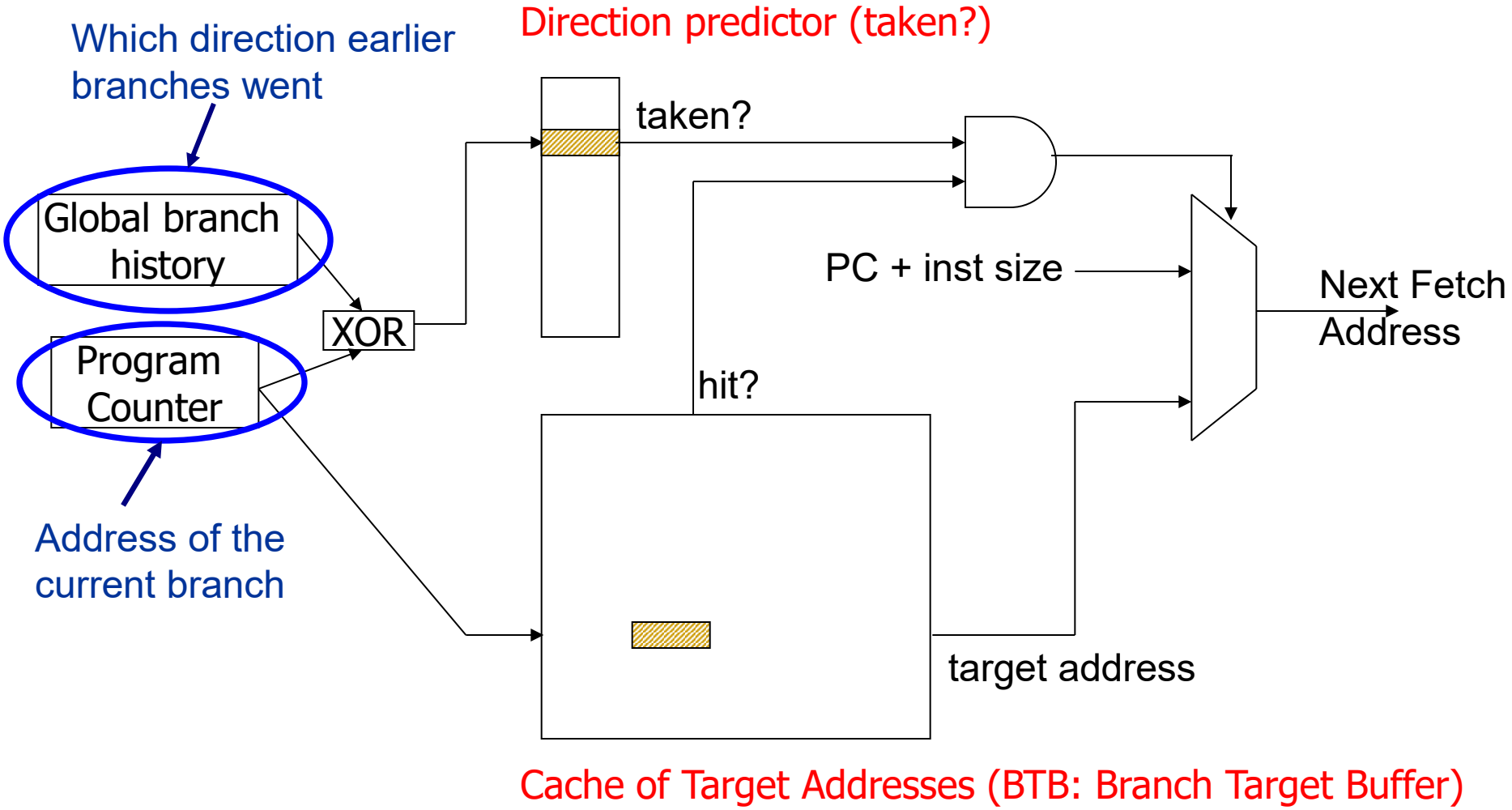
- **观察（Observation）：对于条件直接分支指令而言，其目标地址在不同的动态实例中保持不变。**
  - 思路： 将先前实例中的目标地址存储下来，并与当前的 PC 进行匹配。
  - 这被称为 分支目标缓冲器（BTB, Branch Target Buffer） 或 分支目标地址缓存（Branch Target Address Cache）。

带 BTB 与方向预测的取指阶段



Always taken CPI = [1 + (0.20*0.3) * 2] = 1.12 (70% of branches taken)

更复杂的分支方向预测


Which direction earlier branches went
Direction predictor (taken?)
Global branch history
Program Counter
XOR
taken?
PC + inst size
Next Fetch Address
hit?
Address of the current branch
target address
Cache of Target Addresses (BTB: Branch Target Buffer)

需要预测的三个内容

- **在取指阶段需要预测三项内容：**
  - 获取的指令是否为分支指令
  - （条件）分支方向
  - 分支目标地址（如果被取，即 taken）

需要预测的三个内容

- **在取指阶段需要预测三项内容：**
  - 获取的指令是否为分支指令
  - （条件）分支方向
  - 分支目标地址（如果被取，即 taken）

- **第三项（3.）可以通过 BTB 完成**
  - 记住上次执行该分支时计算出的目标地址。

- **第一项（1.）也可以通过 BTB 完成**
  - 如果 BTB 为当前 PC 提供了一个目标地址，那么该指令必然是分支指令。
  - **或者（Or）**，我们可以在指令缓存/内存中存储“分支元数据（branch metadata）”位 → 在 I-cache 中存储部分解码后的指令。

- **第二项（2.）：我们如何预测分支方向？**

简单的分支方向预测方案

- **编译期（静态）预测**
  - 始终预测为不跳转（not taken）
  - 始终预测为跳转（taken）
  - BTFN（后向分支预测跳转，前向分支预测不跳转）
    Backward taken, forward not taken
  - 基于性能分析的预测（profile based）
    根据程序历史行为推测最可能的方向

- **运行期（动态）预测**
  - 基于上一次结果的预测（single-bit）
    单比特预测器：预测本次方向与上次相同

更高级的分支方向预测

- **编译期（静态）预测**
  - 始终预测为不跳转（not taken）
  - 始终预测为跳转（taken）
  - BTFN（后向分支预测跳转，前向分支预测不跳转）
    Backward taken, forward not taken
  - 基于性能分析的预测（profile based，可能的方向）
  - 基于程序分析的预测（program analysis based，可能的方向）

- **运行期（动态）预测**
  - 基于上一次结果的预测（single-bit）
    单比特方向预测器
  - 基于两比特计数器的预测（two-bit counter based prediction）
  - 两级预测（two-level prediction：全局 vs. 局部）
  - 混合预测器（Hybrid）

静态分支预测（I）

- **始终预测为不跳转（Always not-taken）**
  - 实现简单： 无需 BTB，也不需要分支方向预测
  - 预测准确率低： 约 30 - 40%（对条件分支而言）
  - 注意： 编译器可以重新布局代码，使最可能的路径成为“未跳转（not-taken）”路径 → 从而使该预测更有效

- **始终预测为跳转（Always taken）**
  - 无需方向预测
  - 准确率更高： 约 60 - 70%（对条件分支而言）
    - 后向分支（如循环分支）通常为 taken
    - 后向分支： 目标地址低于当前分支指令的 PC

- **后向跳转、前向不跳转（BTFN: Backward taken, forward not taken）**
  - 将后向（循环）分支预测为跳转，将其他分支预测为不跳转

静态分支预测（II）

- **基于性能分析（Profile-based）**
  - 基本思想：
    编译器通过 性能分析运行（profile run） 来确定每个分支最可能的方向。
    然后将该方向编码为 提示位（hint bit） 存入分支指令格式中。

- **优点（+）**
  - 逐分支预测（per-branch prediction）
    ——比上一张幻灯片介绍的方案更准确
    ——如果性能分析具有代表性，则预测非常准确！
- **缺点（--）**
  - 需要在分支指令格式中增加提示位（hint bits）
  - 预测准确率依赖于分支运行时的动态行为：
    - TTTTTTTTTTNNNNNNNNNN → 50% 准确率
    - TNTNTNTNTNTNTNTNTNTN → 50% 准确率
  - 预测准确率依赖于用于性能分析（profile）的数据集是否具有代表性

静态分支预测（III）

- **基于程序（Program-based，或基于程序分析 Program analysis based）**
  - 基本思想：使用基于程序分析的启发式规则（heuristics），来确定分支的静态预测方向。
  - 示例 1：基于操作码的启发式（opcode heuristic）例如：将 BLEZ（≤0 则跳转）预测为 不跳转（NT），因为许多程序中负整数常用于错误值。
  - 示例 2：基于循环的启发式（loop heuristic）对保护循环执行的分支预测为 跳转，（即认为循环会执行）
  - 示例 3：针对指针和浮点比较：预测为 不相等（not equal）
- **优点（+）**
  - 不需要性能分析（profiling）
- **缺点（--）**
  - 启发式规则可能不具有代表性或效果不佳
  - 需要编译器分析与 ISA（指令集架构）支持（其他静态方法也有相同要求）
- **Ball 和 Larus，Branch prediction for free，PLDI 1993**
  - 20% 的错误预测率

静态分支预测（IV）

- 基于程序员（Programmer-based）
  - 基本思想：程序员直接提供分支的静态预测方向。
  - 实现方式：通过编程语言中的 pragma 指令（编译提示），用来标注某个分支 更可能跳转 或 更可能不跳转。

- 优点（+）
  - 不需要性能分析（profiling）或程序分析
  - 程序员可能比自动分析工具更了解某些分支及其程序行为

- 缺点（--）
  - 需要编程语言、编译器、以及 ISA 的支持
  - 会增加程序员负担？

Pragmas（编译提示指令）

- 基本思想：使用关键字，使程序员能够向程序变换层级（编译器的底层优化阶段）传递提示信息。

- 示例：用于分支预测的 pragma 风格提示
  - if (likely(x)) { ... }
  - if (unlikely(error)) { ... }

- Pragma 还能用于提供其他许多提示与优化
  - 例如：提示某个循环是否可以并行化
  - #pragma omp parallel
  - 说明（Description）
    - omp parallel 指令显式告诉编译器，可以并行化所选的这段代码。

静态分支预测

- **之前介绍的所有静态技术都可以组合使用**
  - 基于性能分析（Profile-based）
  - 基于程序分析（Program-based）
  - 基于程序员（Programmer-based）

- **那么：你会如何将它们组合？**

- **这三类静态技术的共同缺点是什么？**
  - → 无法适应分支行为的动态变化
    - 这一问题可以通过 动态编译器（dynamic compiler） 进行缓解，但能力有限，粒度较粗，而且动态编译器本身也带来额外开销……
  - 什么是动态编译器？
    - Java JIT（即时编译器）、Microsoft CLR（公共语言运行时） 都属于动态编译器

动态分支预测

- **基本思想：**
  - 基于运行时收集的动态信息来预测分支方向。

- **优点**
  - 预测基于分支的执行历史
    - 因此能够适应分支行为的动态变化
  - 不需要静态性能分析（profiling）
    - 不再受“输入数据集是否具有代表性”这一问题的限制

- **缺点**
  - 更复杂，需要额外的硬件支持

上次结果预测器（Last Time Predictor）

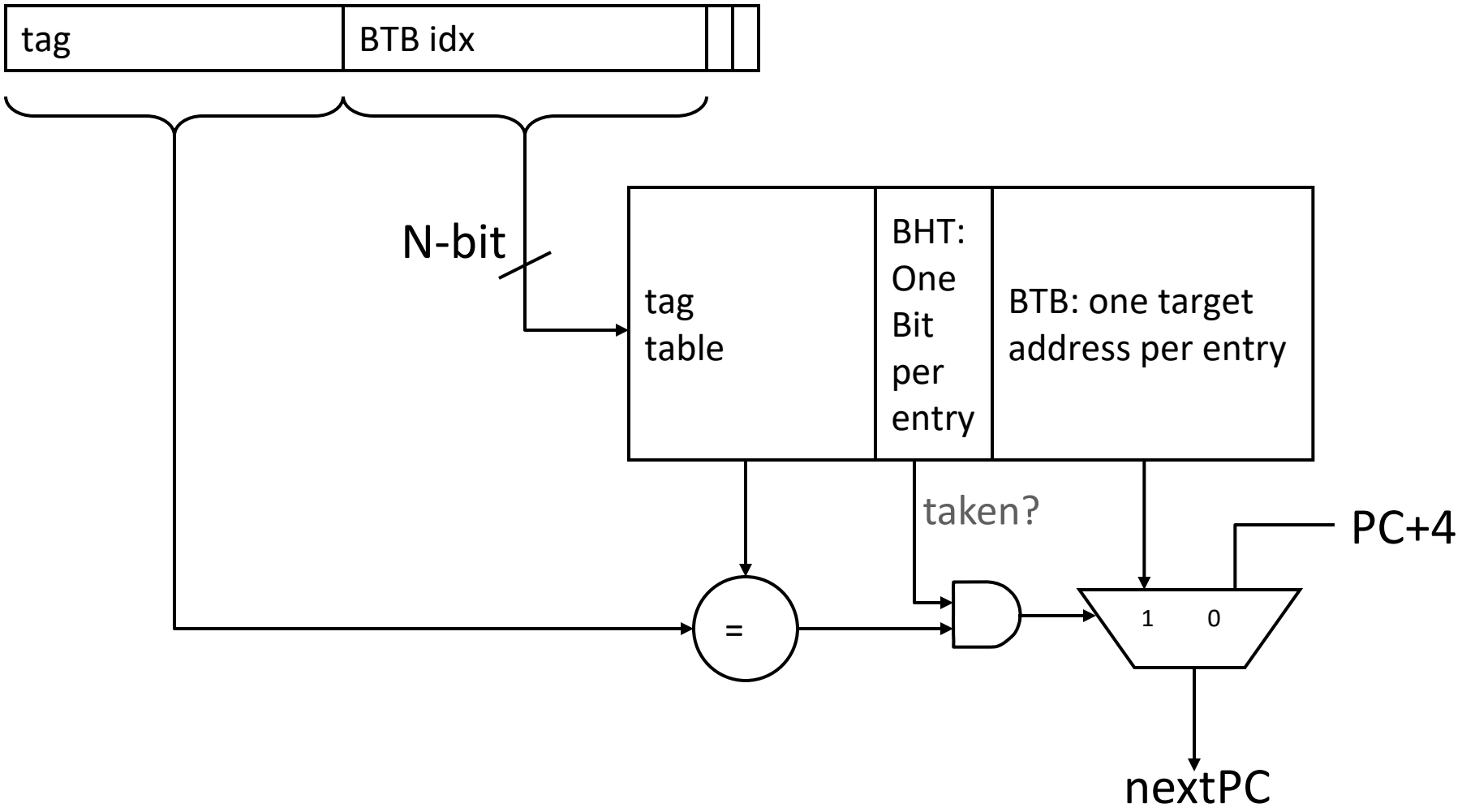
- **上次结果预测器（Last time predictor）**
  - 每个分支使用 1 个比特（存放在 BTB 中）
  - 记录该分支上一次执行时是“跳转”还是“不跳转”
  - TTTTTTTTTTNNNNNNNNNN → 90% 准确率

- **其缺陷：循环分支会在循环的第一次迭代和循环的最后一次迭代必然预测错误**
  - 对于执行 N 次的循环，其准确率为：(N – 2) / N

- **优点（+）对于迭代次数很大的循环（N 大），准确率高**
- **缺点（--）对于迭代次数很小的循环，效果极差**
  例如：TNTNTNTNTNTNTNTNTNTN → 0% 准确率

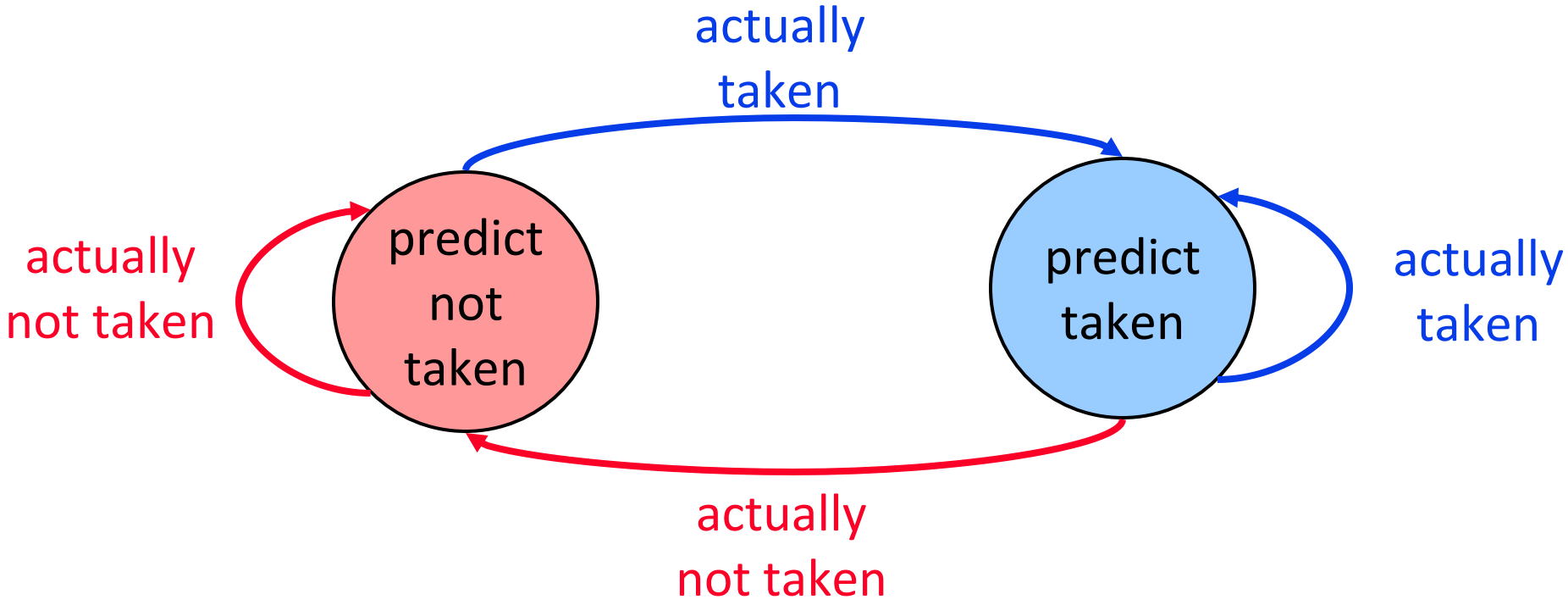
Last-time predictor CPI = $1 + (0.20 \times 0.15) \times 2 = 1.06$（假设预测器准确率为 85%）

实现上次结果预测器（Last-Time Predictor）



- “1-bit BHT（分支历史表）的表项会在每次分支执行后用正确的分支结果进行更新。”
- 预测流程：1. 用 PC 的索引部分访问 tag table、BHT 和 BTB 2. 比较 tag 是否匹配（图中“=”）3. 若 tag 匹配，则使用 BHT 的 1 比特预测。1 → 跳转，0 → 不跳转。4. 若预测为跳转，则 nextPC ← BTB 中的目标地址 5. 若预测为不跳转，则 nextPC ← PC + 4

上次结果预测器的状态机（State Machine for Last-Time Prediction）



改进上次结果预测器（Improving the Last-Time Predictor）

- 问题（Problem）：上次结果预测器会过快地在“跳转（T）→不跳转（NT）”或“NT→T”之间切换预测
  - 即使该分支在大多数情况下是“经常跳转”或“经常不跳转”。
- 解决思路（Solution Idea）：在预测器中加入“滞后性（hysteresis）”，使得预测不会在一次反常结果后立即切换。
  - 使用 两比特（而不是一比特）来记录分支的预测历史
  - 为“跳转”设定两个状态，为“不跳转”也设定两个状态（即 2-bit 饱和计数器）
- Smith, *“A Study of Branch Prediction Strategies,”* ISCA 1981.

基于两比特计数器的分支预测

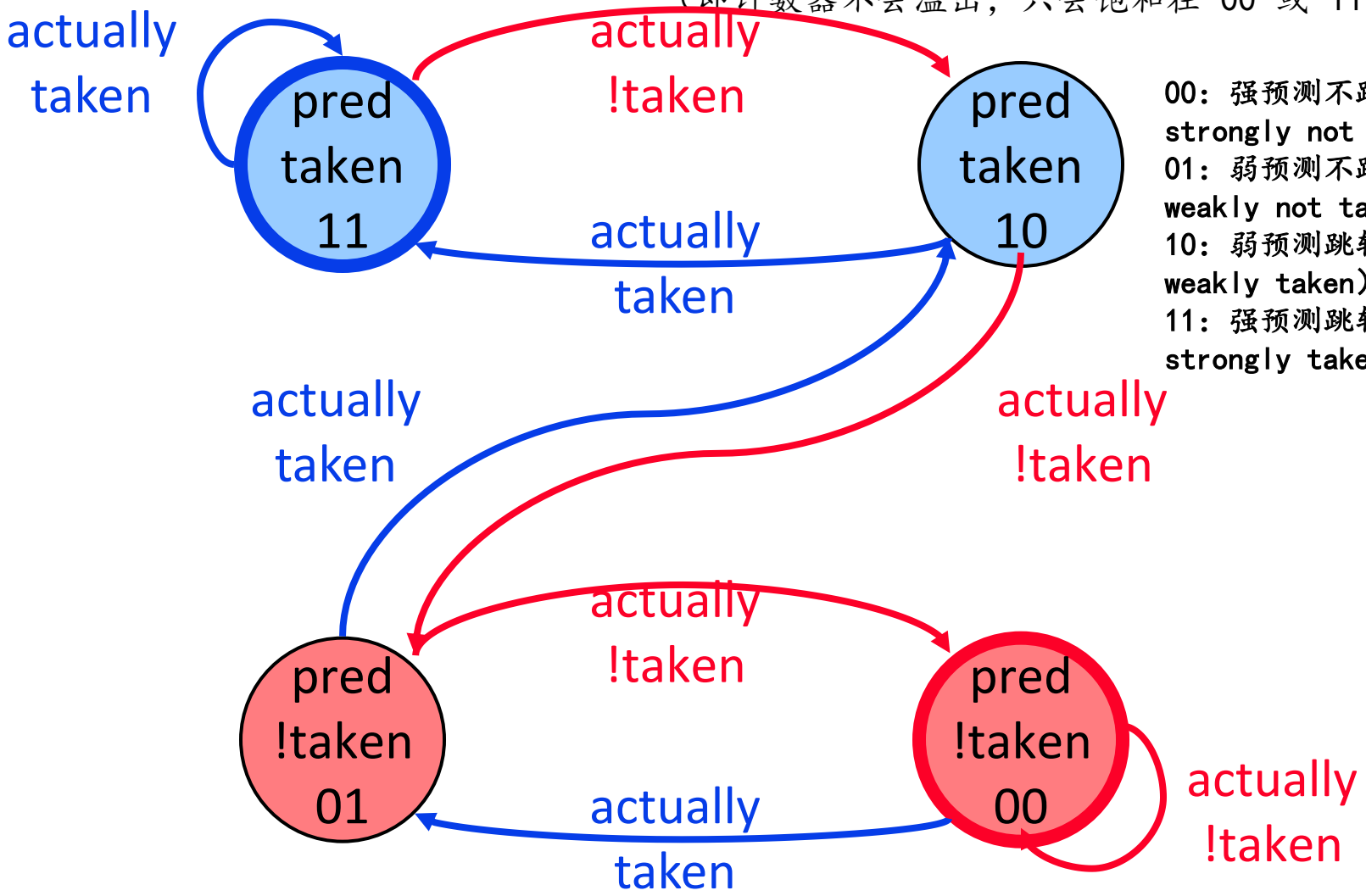
- 每个分支对应一个两比特计数器（2-bit counter）
- 多出的 1 比特提供“滞后性”（hysteresis）
- 强预测不会因为一次反常结果而立即改变方向

- 对于执行 N 次的循环，其预测准确率为：$(N-1)/N$
  - TNTNTNTNTNTNTNTNTNTNT → 50% 准确率（假设计数器初始化为“弱跳转”状态）

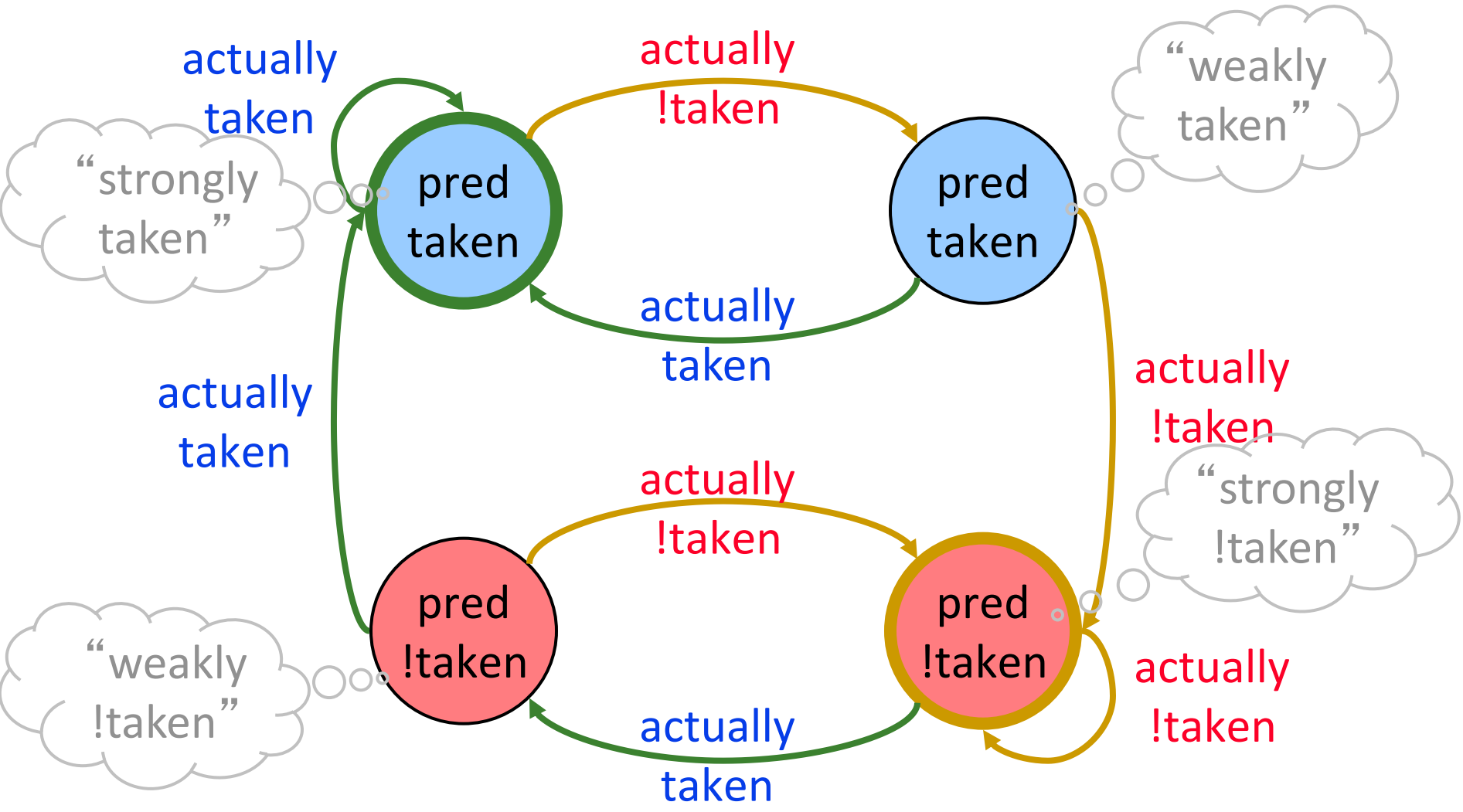
- 优点（+）
  - 更高的预测准确率
  - 2-bit predictor CPI = $1 + (0.20 \times 0.10) \times 2 = 1.04$ （预测准确率约 90%）
- 缺点（--）
  - 需要更多硬件开销，*但计数器可以集成在 BTB 表项中，成本可控*

两比特饱和计数器的状态机（State Machine for 2-bit Saturating Counter）

- 计数器使用“饱和算术”（saturating arithmetic）
  - 算术运算具有最大值与最小值限制
    （即计数器不会溢出，只会饱和在 00 或 11）



使用两比特计数器实现滞后性



- 只有在连续两次预测错误后才会改变预测方向。

这样足够好了吗？

- 基于两比特计数器的预测（也称为双模预测 *bimodal prediction*）在许多程序上能达到约 85% - 90% 的准确率
- 这已经足够好吗？
- 分支预测问题到底有多严重？

重新思考分支问题

- **控制流指令（分支）出现频率很高**
  - 占所有指令的 15% – 25%
- **问题：在流水线处理器中，控制流指令之后的下一条取指地址需要经过 N 个周期才能确定**
  - N 个周期：（最小）分支解析延迟
- **如果处理器每周期取 W 条指令（即流水线宽度为 W）**
  - 一次分支预测错误会导致 N × W 条指令槽被浪费

分支问题的重要性

- **假设条件：**
  - N = 20（20 级流水线 → 分支解析延迟为 20 周期）
  - W = 5（每周期取 5 条指令，5 发射宽度）
  - 每 5 条指令中有 1 条是分支
  - 每个 5 指令块的最后一条为分支
- **问题：取 500 条指令需要多少周期？**
  - 100% 的分支预测准确率
    - 100 周期（所有指令从正确路径取指）
    - 无任何浪费
  - 99% 的分支预测准确率
    - 100（正确路径）+ 20（一次错误路径代价）= **120 周期**
    - 20% 额外指令被取错路径
  - 98% 的分支预测准确率
    - 100（正确路径）+ 20 × 2（两次错误）= **140 周期**
    - 40% 额外指令被取错路径
  - 95% 的分支预测准确率
    - 100（正确路径）+ 20 × 5（五次错误）= **200 周期**
    - 取错路径的指令量 = 100% 的额外指令

我们还能做得更好吗？

- 上次结果预测器（Last-time）与两比特计数器预测器（2BC）利用的是“上次执行结果”的可预测性

- 观察1：一个分支的结果可能与其他分支的结果相关
  - 全局分支相关性（Global branch correlation）

- 观察2：一个分支的结果可能与该分支过去的历史结果相关（不仅仅依赖该分支上次执行时的结果）
  - 局部分支相关性（Local branch correlation）